Daily Paper Review: TAO — Tool-Aware Joint Scheduling for LLM-Based Agentic Serving

Auto-generated Report

October 29, 2026

1 Paper Summary

1.1 Problem and Motivation

Standard LLM serving treats inference requests as self-contained, but agentic sessions *alternate* between GPU-based LLM inference and CPU-based external tool execution (code compilers, web retrievers, file system operations). This creates intermittent GPU demand: sessions occupy resources and KV-cache during inference, release them during tool calls, then need them again upon tool return. Li et al. (2026) identify two deficiencies in existing schedulers: (1) they ignore *when* sessions in tool-execution will return and need inference again, and (2) they rely on per-session greedy heuristics rather than jointly optimizing admission and placement across all concurrent sessions. TAO addresses both by introducing tool-return readiness prediction and dynamic-programming-based joint scheduling [1].

1.2 Method

TAO separates **session evaluation** from **decision making**, enabling independent optimization of each component.

Session evaluation via value triplet. Each candidate session i on GPU node b receives a lexicographic value triplet $(k_{i,b},\, r_i,\, t_i)$: time-aware cache credit (highest priority), tool-return readiness (binary), and token footprint (tiebreaker).

The **time-aware cache credit** decays exponentially to model KV-cache eviction during tool windows:

$$k_{i,b}(t) = k^0_{i,b} \cdot 2^{-\tau_{i,b}/h}, \quad h = 10\text{s}, \tag{1}$$

where $k^0_{i,b}$ is the observed cached-token count and $\tau_{i,b}$ is elapsed time since observation. The 10-second half-life was empirically derived from cache retention patterns on H100 clusters.

Tool-return readiness uses a priority cascade: observed completion > tool-interface deadlines > parsed sleep commands > historical duration statistics with confidence bounds > 5-second default. The predictor achieves 0.41s MAE with 82.9% of predictions within ±0.5s across 10,967 tool calls.

Joint scheduling via dynamic programming. Rather than greedily assigning sessions, TAO formulates admission-placement as a capacity-constrained generalized assignment problem solved via DP:

$$D_j(\mathbf{s}) = \text{lex-max}\big\{D_{j-1}(\mathbf{s}),\ \max_b \big[D_{j-1}(\mathbf{s} - \bar{w}_j \mathbf{e}_b) + \mathbf{u}_{j,b}\big]\big\}, \tag{2}$$

where $D_j(\mathbf{s})$ is the best lexicographic value using sessions $1..j$ with remaining capacity $\mathbf{s}$, $\bar{w}_j$ is the discretized admission cost, and $\mathbf{u}_{j,b}$ is the embedded value triplet. Cache-equivalent nodes

are grouped to reduce dimensionality. The DP solver runs asynchronously to avoid blocking the serving engine, and plans are revalidated against live state at application time, with the longest feasible prefix applied.

1.3 Results

On DeepSeek-V4 Flash with 64 concurrent agent workers on SWE-bench Lite, TAO achieves **26.8% latency reduction** (253.2s → 185.4s), **36.3% TTFT reduction** (0.410s → 0.224s), and **29.5% decode throughput gain** (1561 → 2022 tok/s) over cache-aware vLLM. On GLM-4.7 with SWE-bench Verified (500 tasks): 16.2–18.9% latency reduction, 15.7–38.4% TTFT reduction. Prefix cache hit rates reach 96.31%.

Ablations show the DP joint optimizer is the single most impactful component: replacing it with best-fit-decreasing heuristics causes −13.0% decode throughput. Removing readiness prediction degrades throughput by −3.9% and increases tool-end-to-first-token delay by +11.8%. The DP improved over heuristics in 26.1% of scheduling snapshots, with improvements compounding over long sessions. Total scheduling overhead is <0.12% of runtime.

1.4 Limitations

The DP discretizes capacity, potentially excluding feasible plans in the continuous problem. Tool prediction accuracy (0.41s MAE) is imperfect for a 1-second scheduling horizon, and readiness is binary rather than probabilistic. Only three models on H100 GPUs are tested; no heterogeneous hardware evaluation. On Terminal-Bench (complex tool chains), decode throughput regressed by −8.8% in one configuration. The cache half-life $h = 10$s is fixed rather than adaptive. GPU utilization is not directly measured. The DP improves over heuristics in only ∼26% of snapshots, raising cost-benefit questions for simpler workloads.

1.5 Relevance to My Research

TAO contributes to **LLM inference optimization** (Topic 1) at the orchestration layer. The time-decayed cache credit model advances KV-cache management by treating cache presence as a decaying probability rather than a binary state. The 26.8% latency and 29.5% throughput gains are achieved purely through scheduling—orthogonal to and stackable with model-level optimizations (quantization, pruning, speculative decoding). As LLM agents become more tool-heavy, the fraction of wall-clock time in tool execution grows, making scheduling quality a dominant factor in end-to-end efficiency. TAO provides foundational infrastructure for this emerging agentic serving paradigm.

2 Follow-up Research Directions

2.1 Direction 1: Adaptive Cache Half-Life via Online Learning

Gap: TAO uses a fixed exponential half-life $h = 10$s for cache credit decay, empirically chosen from DeepSeek traces on H100 nodes. Different models, GPU configurations, batch sizes, and workload mixes likely exhibit different cache retention dynamics. A fixed h may be too aggressive (discounting valid cache) or too conservative (overvaluing evicted cache) under varying conditions.

Approach: Replace the fixed h with an online-learned estimate. Maintain a per-node exponential moving average of actual cache hit rates as a function of elapsed time since last access. Fit h by minimizing the cross-entropy between the predicted cache survival probability $2^{-\tau/h}$ and observed binary hit/miss outcomes. Update h every scheduling epoch (e.g., every 100

decisions) via a single-parameter gradient step. Evaluate on workloads with varying tool-call durations (short: <1s, medium: 1–10s, long: >10s) to measure sensitivity.

Feasibility: High. The hit/miss signal is already available from the serving engine. The single-parameter online fit adds negligible computation. The main risk is instability from noisy observations, mitigatable via a prior on $h \in [2, 30]$s and exponential moving average smoothing.

2.2 Direction 2: Probabilistic Readiness with Uncertainty-Aware Scheduling

Gap: Tool-return readiness in TAO is a binary signal, but tool durations are stochastic. A session with 70% probability of returning within the horizon should be valued differently from one with 99% probability. The binary signal loses this information, leading to suboptimal scheduling when multiple sessions have uncertain readiness.

Approach: Replace the binary readiness $r_i \in \{0, 1\}$ with a continuous readiness probability $p_i \in [0, 1]$, estimated from a per-tool-function duration distribution. Model each tool function's duration as a log-normal distribution (a standard choice for latency modeling) fit from historical observations. Compute $p_i = \Pr(\text{duration} \leq \tau_{\text{elapsed}} + H)$ from the fitted CDF. Modify the value triplet to $(k_{i,b},\, p_i,\, t_i)$ and update the DP to handle continuous second-dimension values via discretization into B buckets (e.g., $B = 10$). Measure whether probabilistic readiness improves scheduling quality on workloads with high tool-duration variance.

Feasibility: Moderate. The log-normal fit requires >20 observations per tool function (achievable after warm-up). The DP state space grows by factor B in the readiness dimension, but $B = 10$ keeps it tractable. The main challenge is calibrating the distribution—mispredictions in the tails could cause worse scheduling than the binary heuristic.

2.3 Direction 3: Co-Optimizing Scheduling with Speculative Decoding for Agentic Sessions

Gap: TAO optimizes session-level scheduling but treats inference within each session as a black box. Speculative decoding (using a small draft model to propose tokens verified by the large model) can reduce per-session inference latency, but its effectiveness depends on GPU headroom—speculation is wasteful when the GPU is saturated. Currently, scheduling and speculative decoding are decoupled: the scheduler does not know whether a session would benefit from speculation, and the speculative decoder does not account for cluster-level load.

Approach: Extend TAO's value triplet to a quadruplet $(k_{i,b},\, r_i,\, \sigma_i,\, t_i)$, where $\sigma_i \in \{0, 1\}$ indicates whether session i should use speculative decoding on the current scheduling cycle. The DP jointly optimizes admission, placement, *and* speculation assignment: admitting a session with speculation costs more capacity (draft model overhead) but reduces its expected latency. Define an augmented cost $w_i(\sigma_i) = w_i^{\text{base}} + \sigma_i \cdot w_i^{\text{spec}}$ and value $u_{i,b}(\sigma_i) = u_{i,b}^{\text{base}} + \sigma_i \cdot \Delta u_i^{\text{spec}}$, where Δu_i^{spec} is the expected latency benefit. The DP state doubles (each session has two options), but grouping and pruning keep it tractable.

Feasibility: Moderate. Requires integrating a speculative decoding engine (e.g., Eagle, Medusa) into the serving stack and exposing its resource cost to the scheduler. The augmented DP is straightforward algorithmically. The main challenge is accurately estimating Δu_i^{spec} online, since speculation acceptance rates vary by context.

3 Conclusion

TAO makes a principled contribution to agentic LLM serving by recognizing that tool-execution windows create scheduling opportunities invisible to conventional systems. The time-decayed cache credit, tool-return readiness prediction, and DP-based joint optimization each contribute measurably, with the joint optimizer providing the largest single gain (−13% throughput without it). The system achieves up to 26.8% latency reduction and 29.5% decode throughput

improvement at negligible overhead, demonstrating that orchestration-layer optimization is a powerful and underexplored lever for LLM efficiency—complementary to model-level techniques and increasingly critical as agentic workloads dominate LLM deployment.

References

[1] Shuhuai Li, Shengdong Ye, Dongdong Ge, Yinyu Ye, and Jianghao Lin. TAO: Tool-Aware Joint Scheduling for LLM-Based Agentic Serving. *arXiv preprint arXiv:2610.37230*, 2026.